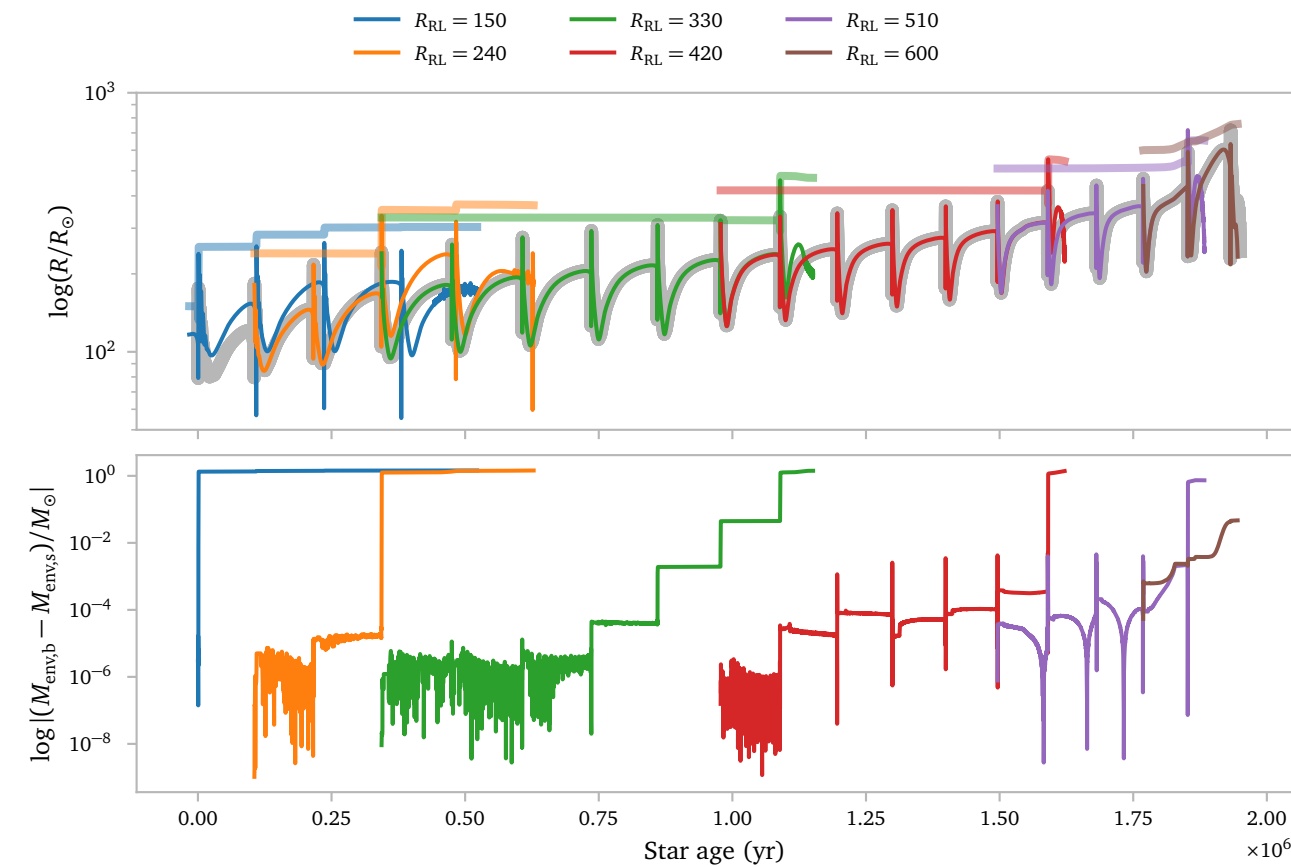


Notes Week 8

a. Selecting Starting models Efficiently

From previous binary TPAGB models, it became clear that a lot of the computational time was spent recomputing parts of the single star evolution, without any noticeable effects from the binary companion. It can therefore be a worthwhile investigation to look at the exact moment where the binary companion starts to significantly impact the donor star and try to select a new starting model criterion from this.

Below, I plot the radius of the $\text{\AE S O P U S}$ models from last week for $q = 0.75$ and different starting orbital separations. I also show the difference in envelope *mass* for the reference single star model, and the binary model.



I also made similar plots for the other q values, which can be found at the end of this document. By taking a maximum $\Delta M = |M_{\text{sin}} - M_{\text{bin}}|$ of about $0.01M_{\odot}$, we can see that in most cases the star only exceeds this maximum after the first Roche lobe overflowing thermal pulse. Sometimes however, the first pulse can *fail*, as can be seen for the $R_{\text{RL}} = 330R_{\odot}$ model. Then There are two pulses where the star exceeds the maximum.

I therefore wrote the following starting model selector:

```
def find_model(R):
    searching_TP = True
    # search models in order of descending radius
    for key in list(models_dict.keys())[::-1]:
        model = models_dict[key]

        # searching for the thermal pulse count
        # of the first presumable RLOF occurence
        if searching_TP:
            if model["R"] < 0.95 * R:
                TP_collision = model["TP"]
                searching_TP = False

        # after finding the thermal pulse count,
        # finding a model 2 thermal pulses earlier
        else:
            print(model["TP"], model["R"])
            if model["TP"] <= (np.max([TP_collision - 2, 0])) and model["R"] < 0.9 * R:
                return model
```

This basically ensures the starting model occurs at least 2 thermal pulses before the Roche lobe overflowing thermal pulse, and has a starting radius smaller than $0.9R_{\text{RL}}$.

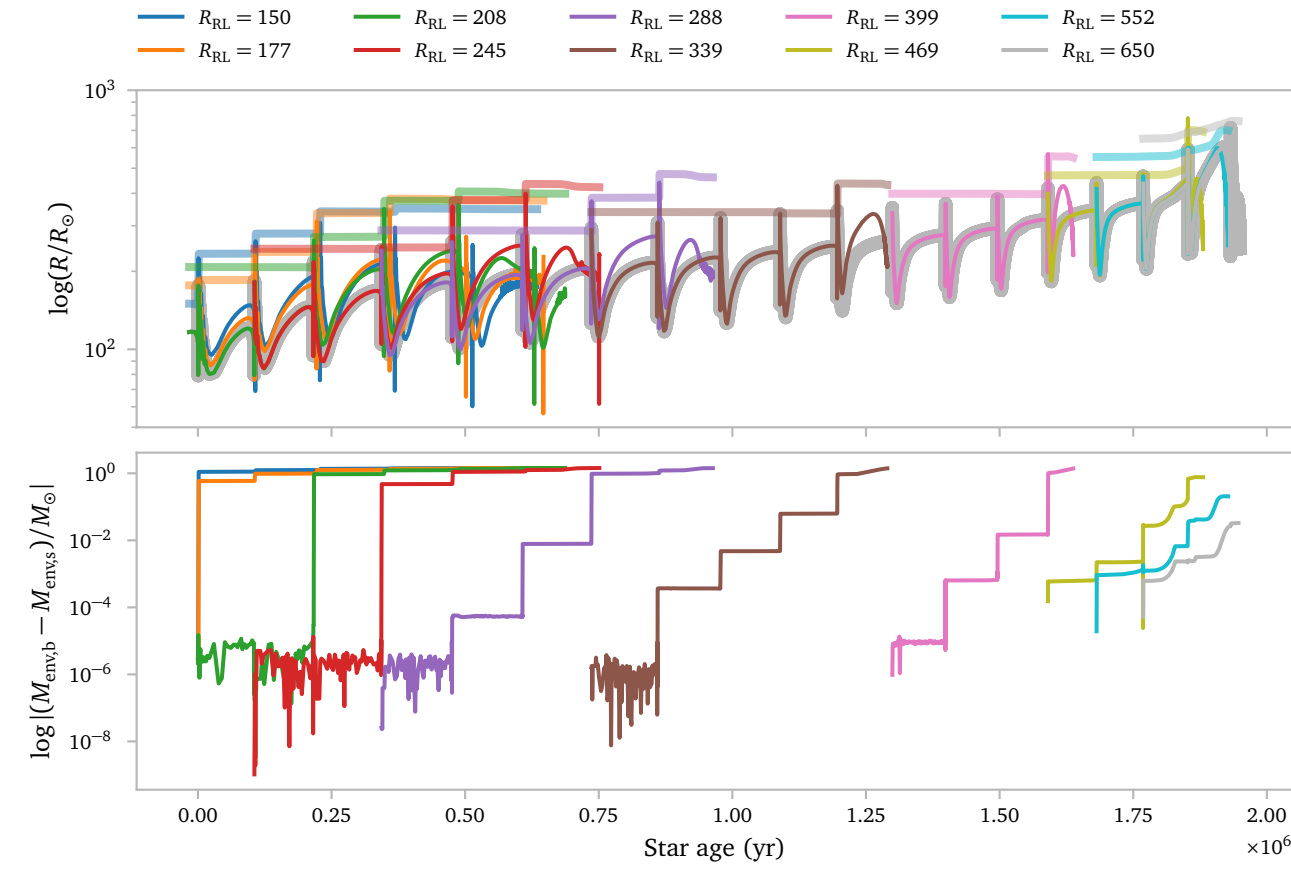
I use this new implementation for the next grid of models. I also changed it so the grid uses initial q instead of q at the moment the accretor gets added¹.

¹ However, I forgot to change this in the grid of models I ran, so these still use the other q .

b. A new Grid

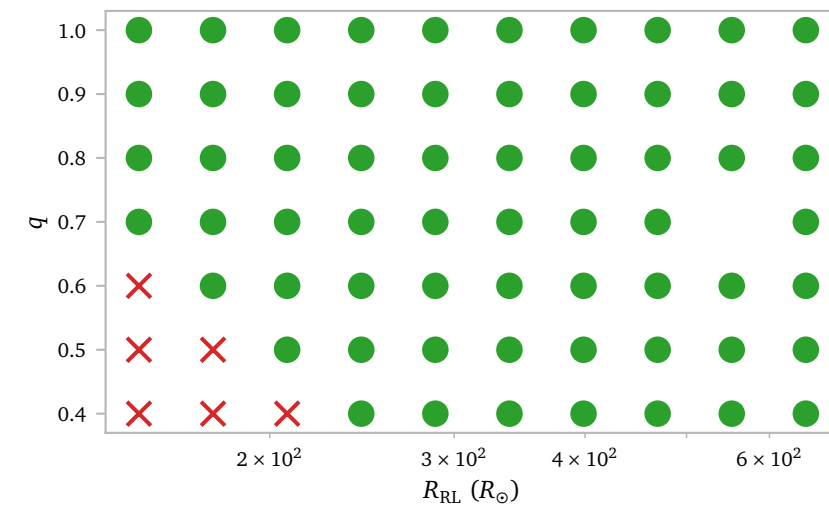
I ran models with 10 log spaced R_{RL} values from 150 to 650 $R_{\odot}$ and q from 0.4 to 1 in steps of 0.1.

As a comparison, here is the same plot as I showed above, but now for the new grid and $q = 0.8$:



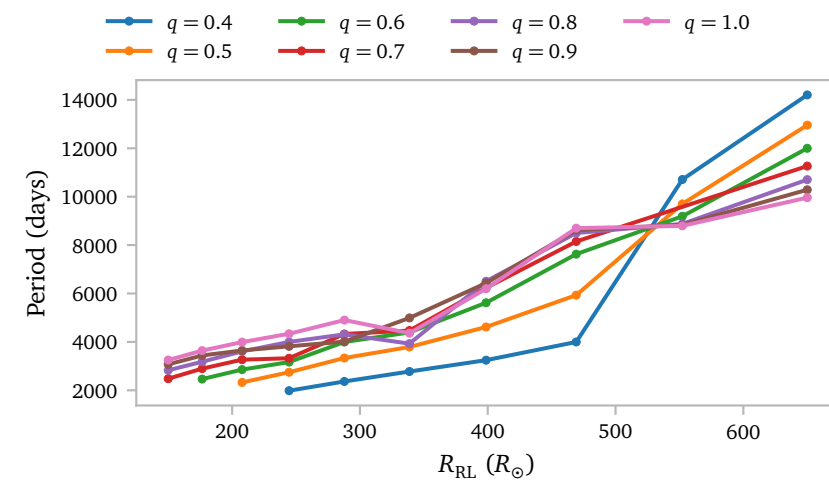
The simulation times before the onset of significant mass transfer are a bit shorter. Maybe an even more aggressive setting can be used though.

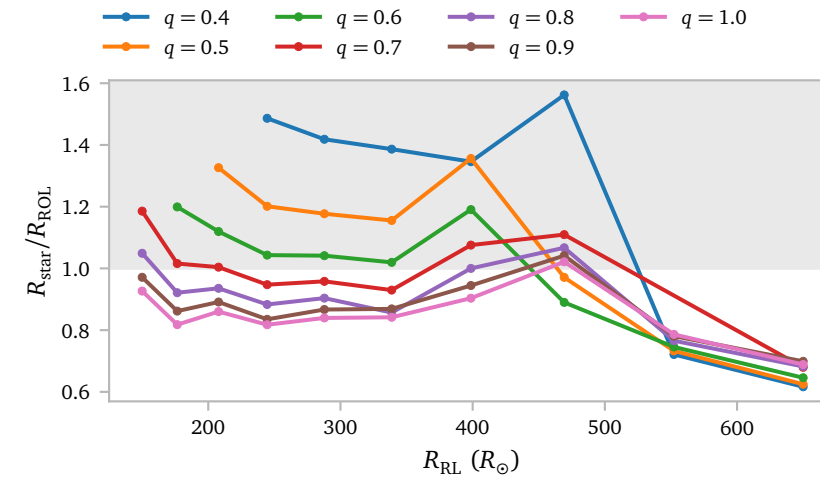
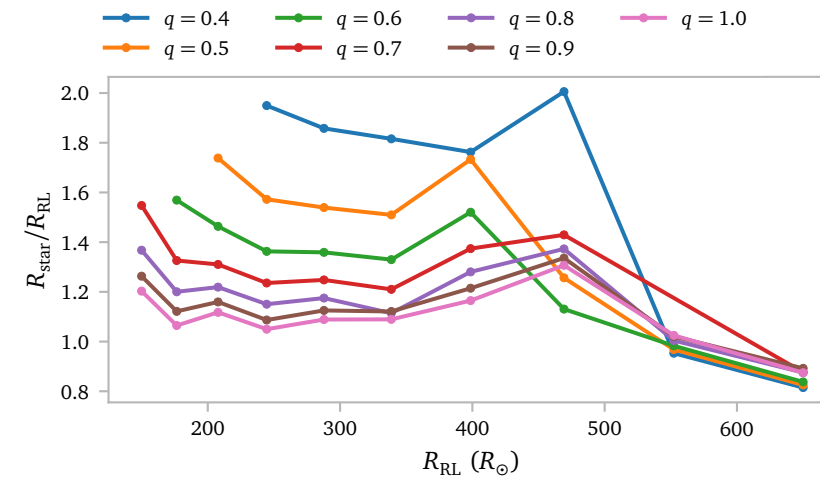
Below, I show which models ran successfully:



There is a model missing at $R_{\text{RL}} = 552.27$ and $q = 0.7$. This model did not finish its evolution in 2 days. The results are still in the scratch directory of the node at which the model ran. I would need to manually move it to my work directory which I have not done yet.

Below, I show the final periods for the models:





I am not sure which other plots are interesting to make, except for looking at the individual models.

c. Tidal Synchronization

c.1 Theory

Here, I describe the (I think) relevant results from [Hut \(1981\)](#) and [Preece et al. \(2022\)](#). In particular, I describe the tidal evolution equations (9), (10) and (11) from [Hut \(1981\)](#):

$$\begin{aligned}\frac{da}{dt} &= -6 \frac{k}{T} q(1+q) \left(\frac{R}{a}\right)^8 \frac{a}{(1-e^2)^{15/2}} \left[f_1(e^2) - (1-e^2)^{3/2} f_2(e^2) \frac{\Omega}{n} \right], \\ \frac{de}{dt} &= -27 \frac{k}{T} q(1+q) \left(\frac{R}{a}\right)^8 \frac{e}{(1-e^2)^{13/2}} \left[f_3(e^2) - \frac{11}{18} (1-e^2)^{3/2} f_4(e^2) \frac{\Omega}{n} \right], \\ \frac{d\Omega}{dt} &= 3 \frac{k}{T} \frac{q^2}{r_g^2} \left(\frac{R}{a}\right)^6 \frac{n}{(1-e^2)^6} \left[f_2(e^2) - (1-e^2)^{3/2} f_5(e^2) \frac{\Omega}{n} \right].\end{aligned}$$

Here, f_i is a family of functions only dependent on e^2 which a nice similarity, namely that when $e = 0$, $f_i = 1$. Since we are (at least now,) only looking at circular orbits ($e = 0$), the equations above simplify to

$$\frac{da}{dt} = -6 \frac{k}{T} q(1+q) \left(\frac{R}{a}\right)^8 a \left[1 - \frac{\Omega}{n} \right], \quad (1)$$

$$\frac{de}{dt} = 0, \quad (2)$$

$$\frac{d\Omega}{dt} = 3 \frac{k}{T} \frac{q^2}{r_g^2} \left(\frac{R}{a}\right)^6 n \left[1 - \frac{\Omega}{n} \right]. \quad (3)$$

In these equations, n^2 is the mean orbital angular velocity and r_g is the radius of gyration³.

A last important factor in these equations is the ratio k/T , which contains the dependence of the equations on the structure of the star ([Preece et al., 2022](#)). Here, the k is the *apsidal motion* constant⁴ and T is the tidal response time.

This ratio can be approximated following [Preece et al. \(2022\)](#) as

$$\left(\frac{k}{T}\right)_{\text{conv}} = \left(\frac{R_{\text{conv}}}{R}\right)^a \left(\frac{M_{\text{conv}}}{M}\right)^b \frac{c}{t_{\text{conv}}}, \quad (4)$$

where a , b and c are functions of the metallicity z only, and t_{conv} is given by

$$t_{\text{conv}} = 0.4311 \left[\frac{3M_{\text{conv}} R_{\text{conv}}^2}{L} \right]^{1/3} \text{ yr.} \quad (5)$$

For completeness, I also list the equations for a , b and c below as well:

$$a \approx 2.72 + 0.63 \cdot \log z,$$

$$b \approx 0.68 - 0.219 \cdot \log z,$$

$$c \approx 0.12 - 0.023 \cdot \log z.$$

Lastly, a correction factor f_{conv} for *fast tides*⁵ can be used to change k/T in the fast tide regime.

There exist multiple prescriptions for choosing sensible values for f_{conv} . I only describe the prescription by [Goldreich & Nicholson \(1989\)](#) as it is implemented in *Binary Star Evolution* (BSE) ([Hurley et al., 2002](#)):

$$f_{\text{conv}} = \min \left[1, \left(\frac{P_{\text{tid}}}{2t_{\text{conv}}} \right)^2 \right],$$

where P_{tid} is the tidal pumping timescale given by

$$\frac{1}{P_{\text{tid}}} = \left| \frac{1}{P_{\text{orb}}} - \frac{1}{P_{\text{spin}}} \right|.$$

In the case of a radiative envelope, the ratio k/T should be damped, but since TPAGB stars only have convective envelopes, there is (at least at this point) no need to describe and implement this damping.

² $n \equiv G^{1/2}(M+m)^{1/2}a^{-3/2}$

³ r_g is defined as $I = M(r_g R)^2$, where I the moment of inertia of the primary.

⁴ Does this relate to how much an elliptical orbit will rotate (apsidal precession)?

⁵ Regime where the tidal period (what is the tidal period?) is shorter than the convective turnover time t_{conv} .

c.2 MESA Toy Model

MESA has an implementation for tides in binary stars. This option has to be turned on in the binary inlist settings using `do_tidal_sync`. The implementation of the tidal interactions is interiorly written in the file `$MESA/binary/private/binary_tides.f90` and largely follows the theory described above.

There are, however, small differences. To start, MESA does not use the fit for k/T from Preece et al. (2022), but a similar fit from Hurley et al. (2002). Then, MESA actually only directly computes the change of the spin angular frequency of the star(s). By keeping track of the lost or gained angular momentum, the orbital separation and eccentricity are then adjusted using conservation of angular momentum elsewhere. Below, I show the piece of code that is responsible for adjusting the spin angular momentum in the default implementation of tides in MESA:

```
! binary_tides.f90
subroutine sync_spin_to_orbit(...)
...
  if (sync_mode == "Uniform") then
    a1 = f2(b% eccentricity)
    a2 = pow(1-pow2(b% eccentricity), 1.5d0)*f5(b% eccentricity)
    do k=1,nz
      delta_j(k) = (1d0 - exp(-a2*dt_next/t_sync))*(s% j_rot(k) - a1/a2*j_sync(k))
    end do
...

  if (.not. b% doing_first_model_of_run) then
    do k=1,nz
      s% extra_jdot(k) = s% extra_jdot(k) - delta_j(k)/dt_next
    end do
  end if

end subroutine sync_spin_to_orbit
```

The `s% extra_jdot(k)` is used to spin the star up or down. In most cases, this is exactly what we want, but in our case, where we want to only see the effects of the tides, and preferably keep a naive non-rotating star, this is not what we want.

To only affect the *orbit*, we need to bypass the star interiorly and update the orbital angular momentum directly. I think the easiest way to do is to utilize the `other_jdot_ls` hook. By enabling this other hook in the binary inlist, and writing the following line in the `extras_run_binary_controls`:

```
b% other_jdot_ls => toy_tides
```

we can now implement our own tide formulation. As the name of the `other_jdot_ls` hook reveals, my first try will just be a very simple toy example. Below, I write down my code:

```
! src/bin/additional_routines.inc
subroutine toy_tides(binary_id, ierr)
! --- following the required input and output of default_jdot_ls
  integer, intent(in) :: binary_id
  integer, intent(out) :: ierr

! --- getting access to the binary object and star object
  type(binary_info), pointer :: b
  type(star_info), pointer :: s

! --- necessary other variables
  real(dp) :: omega, omega_orb, domega_dt
  real(dp) :: I_eff, Jdot_tides, dt
  real(dp) :: t_sync
  integer :: k ! for counting zones

  ierr = 0
  call binary_ptr(binary_id, b, ierr)
  s => b% s_donor

  dt = s% dt

! --- get initial spin and orbital frequency
  omega = s% x_ctrl(3)
  omega_orb = 2d0*pi / b% period

! --- arbitrary sync timescale
  t_sync = 1d12 ! in seconds (30000 yr)

! --- simplified hut spin evolution
  domega_dt = (omega_orb - omega) / t_sync

! --- update spin
  omega = omega + dt * domega_dt
  s% x_ctrl(3) = omega

! --- compute moment of inertia of solid body
  I_eff = 0
  do k=1,s% nz
    I_eff = I_eff + s% dm_bar(k) * (s% r(k)**2)
  end do
  Jdot_tides = I_eff * domega_dt

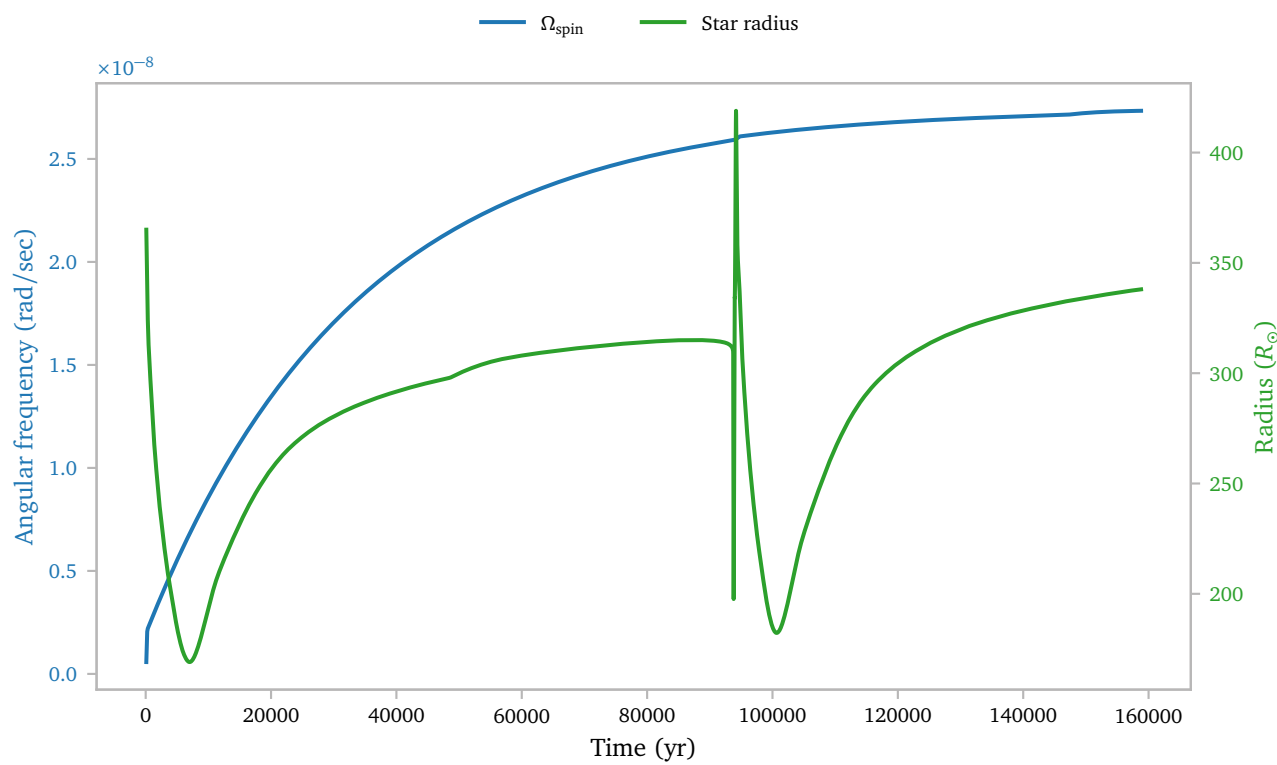
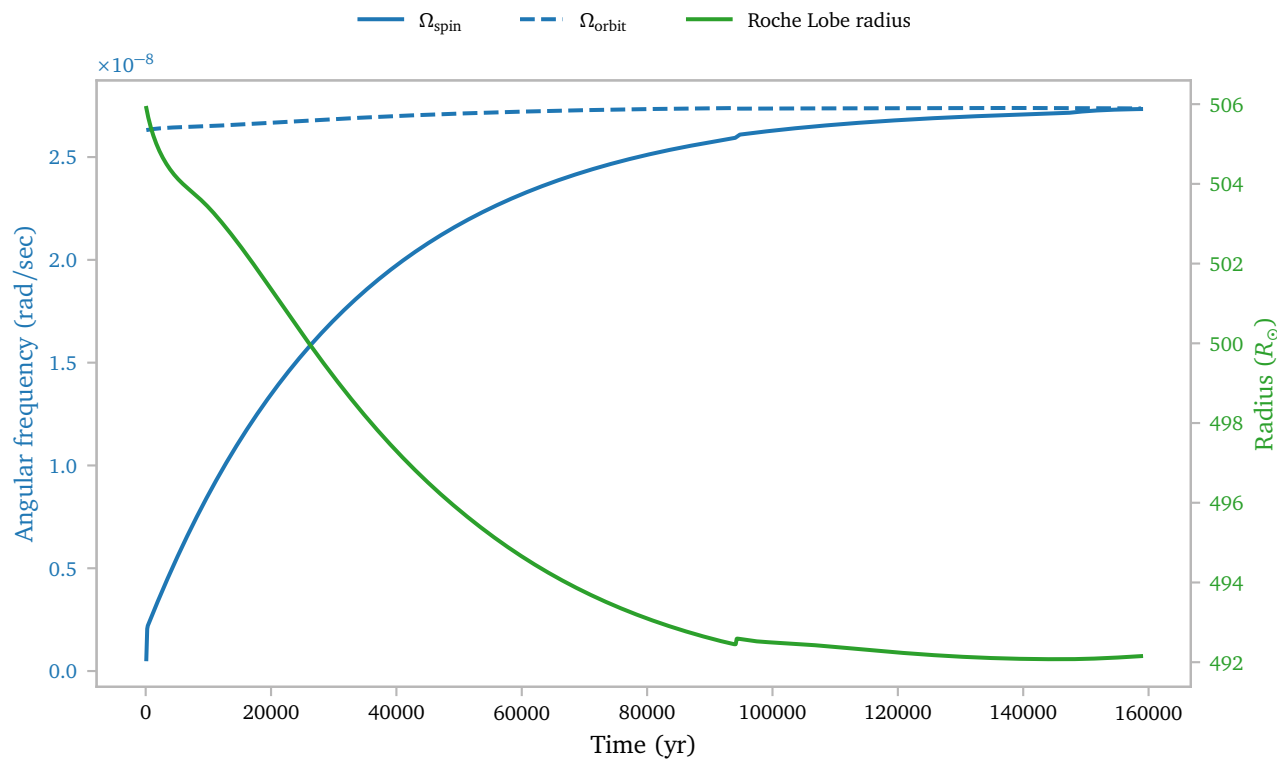
! --- change jdot orbit
  b% jdot_ls = - Jdot_tides

! --- write orbit and spin frequency
  write(*,*) "Omega_star = ", omega
  write(*,*) "Omega_orb = ", omega_orb
  write(*,*) "J_orb =", b% angular_momentum_j
  write(*,*) "a =", b% separation

end subroutine toy_tides
```

I also add the spin angular frequency and the orbit angular frequency to the history columns to visualize the effect of the tides.

It is important to realize that the toy model is an extremely simplified version of a full tidal implementation. Below, I plot the results of the simple model. The star is initially non-rotating.



It is obvious that the simple tidal model works in the sense that the difference between the orbital angular frequency and spin angular frequency approach eachother. Since the star starts out with a slower spin, the orbit decreases (and thus the Roche lobe radius too). The second plot however, shows that the spin of the star at this point, is completely unaffected by changes in the structure of the star. During a radius peak, conservation of angular momentum should cause the star to show down its rotation, but this does not happen.

I think this is nice and easy improvement to make to the toy model:

c.3 MESA Toy Model with Stellar Evolution

```
! src/bin/additional_routines.inc
subroutine toy_tides_with_star_evo(binary_id, ierr)
  ! --- following the required input and output of default_jdot_ls
  integer, intent(in) :: binary_id
  integer, intent(out) :: ierr

  ! --- getting access to the binary object and star object
  type(binary_info), pointer :: b
  type(star_info), pointer :: s

  ! --- necessary other variables
  real(dp) :: omega, omega_orb, domega_dt
  real(dp) :: I_eff, Jdot_tides, dt
  real(dp) :: I_old, dlnI_dt, t_sync
  real(dp) :: domega_dt_tides

  integer :: k ! for counting zones

  ierr = 0
  call binary_ptr(binary_id, b, ierr)
  s => b% s_donor

  dt = s% dt

  ! --- get initial spin and orbital frequency
  omega = s% x_ctrl(3)
  omega_orb = 2d0*pi / b% period

  ! --- arbitrary sync timescale
  t_sync = 1d12

  ! --- compute moment of inertia of solid body
  I_eff = 0
  do k=1,s% nz
    I_eff = I_eff + s% dm_bar(k) * (s% r(k)**2)
  end do

  ! --- compute dlnI/dt
  I_old = s% x_ctrl(4)
  if (I_old > 0d0 .and. dt > 0d0) then
    dlnI_dt = (log(I_eff) - log(I_old)) / dt
  else
    dlnI_dt = 0d0
  end if

  ! --- simplified hut spin evolution
  domega_dt_tides = (omega_orb - omega) / t_sync

  ! --- total spin evolution (structure + tides)
  domega_dt = domega_dt_tides - omega * dlnI_dt

  ! --- update spin
  omega = omega + dt * domega_dt
  s% x_ctrl(3) = omega

  ! --- store I for next step
  s% x_ctrl(4) = I_eff

  ! --- change jdot orbit
  Jdot_tides = I_eff * domega_dt_tides
  b% jdot_ls = - Jdot_tides

  ! --- write orbit and spin frequency
  write(*,*) "Omega_star = ", omega
  write(*,*) "Omega_orb = ", omega_orb
  write(*,*) "J_orb =", b% angular_momentum_j
  write(*,*) "a =", b% separation
end subroutine toy_tides_with_star_evo
```

The only change with respect to the previous version is the addition of dlnI/dt and the separation of the tides.

Below, I write a quick derivation of domega_dt = - omega * dlnI_dt, where I assume no tides. If there are no tides, then

$$\frac{dJ}{dt} = \frac{d}{dt} (I\Omega) = I \frac{d\Omega}{dt} + \Omega \frac{dI}{dt} = 0.$$

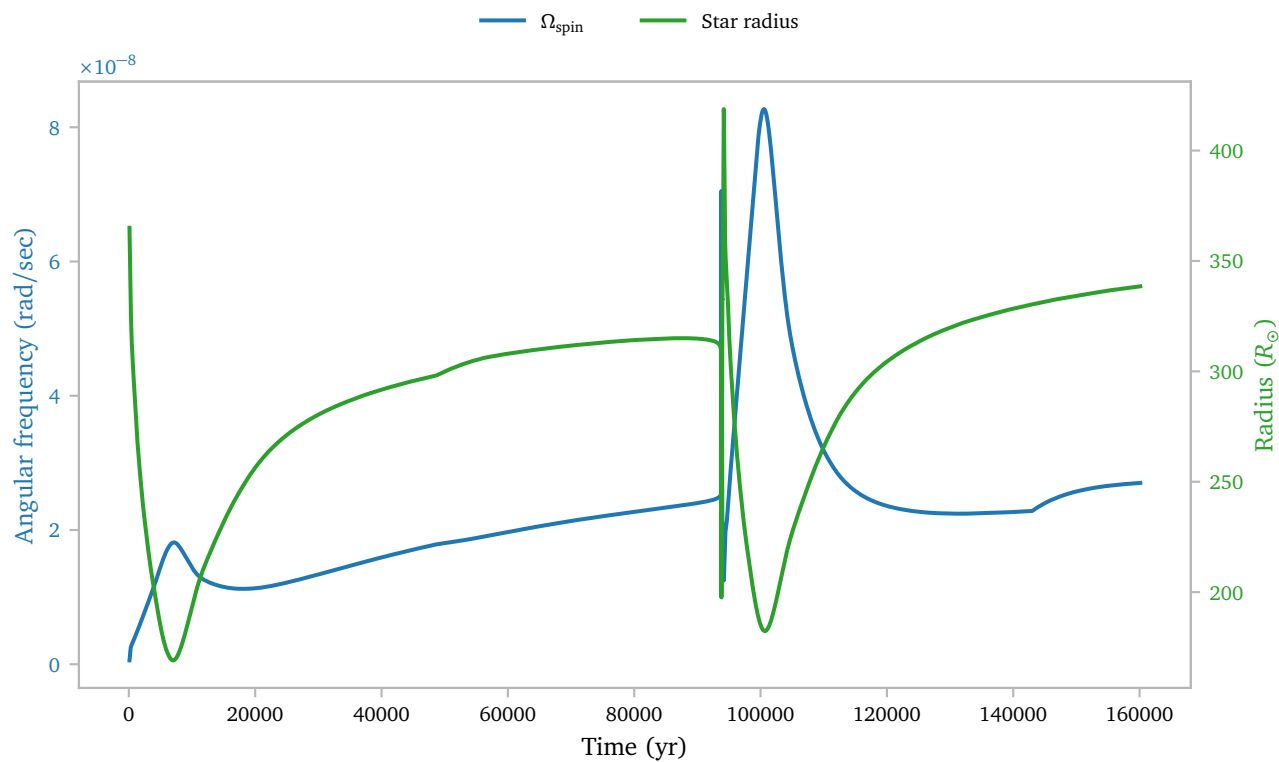
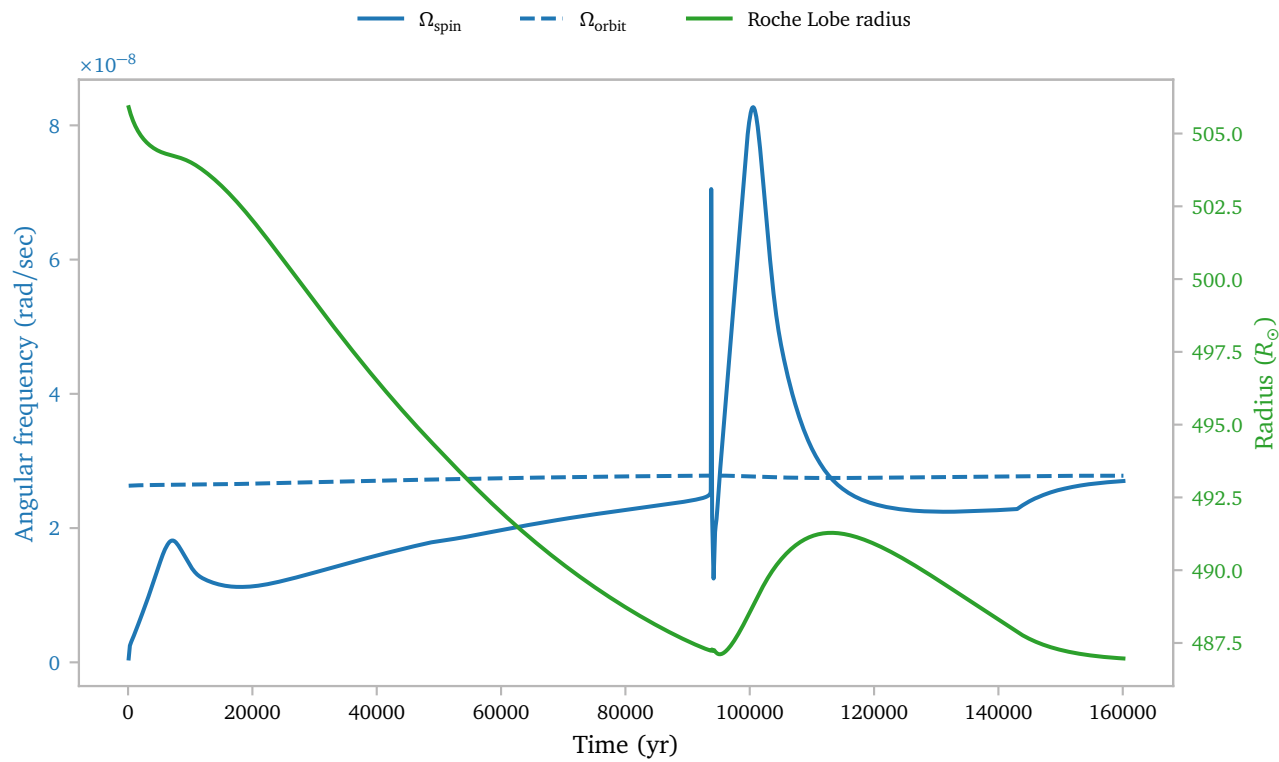
Rearranging gives

$$\frac{d\Omega}{dt} = -\Omega \frac{1}{I} \frac{dI}{dt} = -\Omega \frac{d \ln I}{dt}.$$

Obviously, this also means that

$$\frac{d\Omega}{dt} = \left(\frac{d\Omega}{dt} \right)_{\text{tides}} - \Omega \frac{1}{I} \frac{dI}{dt} = -\Omega \frac{d \ln I}{dt}.$$

Below, I show the same plots as I did for the first toy model.



Quite a stark difference. The spin nicely reacts to the change in radius of the star, exactly as I would expect. However, the implementation of the actual tides is still really simplified. In the next subsection, I describe a more sophisticated approach.

c.4 MESA Tidal Model without correction factor

The bulk of the tides subroutine stays the same:

```
subroutine tides(binary_id, ierr)
! --- following the required input and output of default_jdot_ls
integer, intent(in) :: binary_id
integer, intent(out) :: ierr

! --- getting access to the binary object and star object
type(binary_info), pointer :: b
type(star_info), pointer :: s_d
type(star_info), pointer :: s_a

! --- necessary other variables
real(dp) :: omega, omega_orb, domega_dt
real(dp) :: I_eff, Jdot_tides, dt
real(dp) :: I_old, dlnI_dt, rg2
real(dp) :: domega_dt_tides, q, R
real(dp) :: a, k_over_T

integer :: k ! for counting zones

ierr = 0
call binary_ptr(binary_id, b, ierr)
s_d => b% s_donor

dt = s_d% dt

! --- compute mass ratio, R and a
q = b% m(2) / b% m(1)
R = s_d% photosphere_r * Rsun ! in cm
a = b% separation ! in cm

! --- compute k_over_T
k_over_T = compute_k_over_T(s_d)

! --- get initial spin and orbital frequency
omega = s_d% x_ctrl(3)
omega_orb = 2d0*pi / b% period

! --- compute moment of inertia of solid body and gyration radius
I_eff = compute_I_eff(s_d)
rg2 = compute_rg2(s_d, I_eff)

! --- compute dlnI/dt
I_old = s_d% x_ctrl(4)
if (I_old > 0d0 .and. dt > 0d0) then
    dlnI_dt = (log(I_eff) - log(I_old)) / dt
else
    dlnI_dt = 0d0
end if

! --- hut spin evolution
domega_dt_tides = 3* k_over_T * q**2 / (rg2) * (R / a)**6 * (omega_orb - omega)

! --- total spin evolution (structure + tides)
domega_dt = domega_dt_tides - omega * dlnI_dt

! --- update spin
omega = omega + dt * domega_dt
s_d% x_ctrl(3) = omega

! --- store I for next step
s_d% x_ctrl(4) = I_eff

! --- change jdot orbit
Jdot_tides = I_eff * domega_dt_tides
b% jdot_ls = - Jdot_tides

! --- write orbit and spin frequency
write(*,*) "Omega_star = ", omega
write(*,*) "Omega_orb = ", omega_orb
write(*,*) "J_orb =", b% angular_momentum_j
write(*,*) "a =", b% separation
write(*,*) "dt=", dt
write(*,*) "I_eff =", I_eff
write(*,*) "rg2 =", rg2
end subroutine tides
```

However, the routine now calls some functions to compute some necessary variables, starting with compute_I_eff. This function is quite simple:

```
real(dp) function compute_I_eff(s) result(I_eff)
implicit none

type(star_info), pointer :: s
integer :: k

I_eff = 0d0
do k = 1, s% nz
    I_eff = I_eff + (2d0/3d0) * s% dm_bar(k) * s% r(k)**2
end do

end function compute_I_eff
```

Here, I have now added the factor 2/3 because we are summing mass shells and not point masses. Computing r_g^2 is then almost trivial:

```
real(dp) function compute_rg2(s, I_eff) result(rg2)
implicit none

type(star_info), pointer :: s
integer :: k
real(dp) :: I_eff, M, R

M = s% mstar
R = s% r(1)

if (M > 0d0 .and. R > 0d0) then
    rg2 = I_eff / (M * R * R)
else
    rg2 = 0d0
end if

end function compute_rg2
```

The only thing that is important to realise is that r(k) goes from the outside to the inside, such that r(1) is the radius of the outside layer.

Then I defined compute_k_over_T, based on Preece et al. (2022):

```
real(dp) function compute_k_over_T(s) result(k_over_T)
implicit none

type(star_info), pointer :: s
real(dp) :: a, b, c
real(dp) :: log_z
real(dp) :: R_conv, R
real(dp) :: M_conv, M
real(dp) :: t_conv

log_z = log10(0.014)
a = 2.72 + 0.63 * log_z
b = 0.68 - 0.219 * log_z
c = 0.12 - 0.023 * log_z

M_conv = compute_M_conv(s)
M = s% star_mass

R_conv = compute_R_conv(s)
R = s% photosphere_r

t_conv = compute_t_conv(s, M_conv, R_conv)

k_over_T = (R_conv / R)**a * (M_conv / M)**b * c/t_conv

end function compute_k_over_T
```

This function is really simple, but it does call three other functions, compute_M_conv, compute_R_conv and compute_t_conv. I write them on the next page.

```
real(dp) function compute_M_conv(s) result(M_conv)
implicit none

type(star_info), pointer :: s
integer :: i
real(dp) :: q_top, q_bot

M_conv = 0
do i = 1, s% n_conv_regions
    if (abs(s% cz_top_mass(i) / Msun / s% star_mass - 1d0) < 3d-2) then
        ! --- this is the envelope
        q_top = s% cz_top_mass(i) / Msun / s% star_mass
        q_bot = s% cz_bot_mass(i) / Msun / s% star_mass

        M_conv = s% mstar * (q_top - q_bot) / Msun
    end if
end do

end function compute_M_conv

real(dp) function compute_R_conv(s) result(R_conv)
implicit none

type(star_info), pointer :: s
real(dp) :: R_bot, q_bot, R_top
integer :: k, i

R_conv = 0
R_top = s% r(1)

! --- first find bottom q
do i = 1, s% n_conv_regions
    if (abs(s% cz_top_mass(i) / Msun / s% star_mass - 1d0) < 3d-2) then
        ! --- this is the envelope
        q_bot = s% cz_bot_mass(i) / Msun / s% star_mass
    end if
end do

! --- now find which r this corresponds to
R_bot = 0d0 ! for safety

do k = 1, s% nz
    if (s% m(k) / Msun / s% star_mass <= q_bot) then
        R_bot = s% r(k)
        exit
    end if
end do

R_conv = (R_top - R_bot) / Rsun

end function compute_R_conv

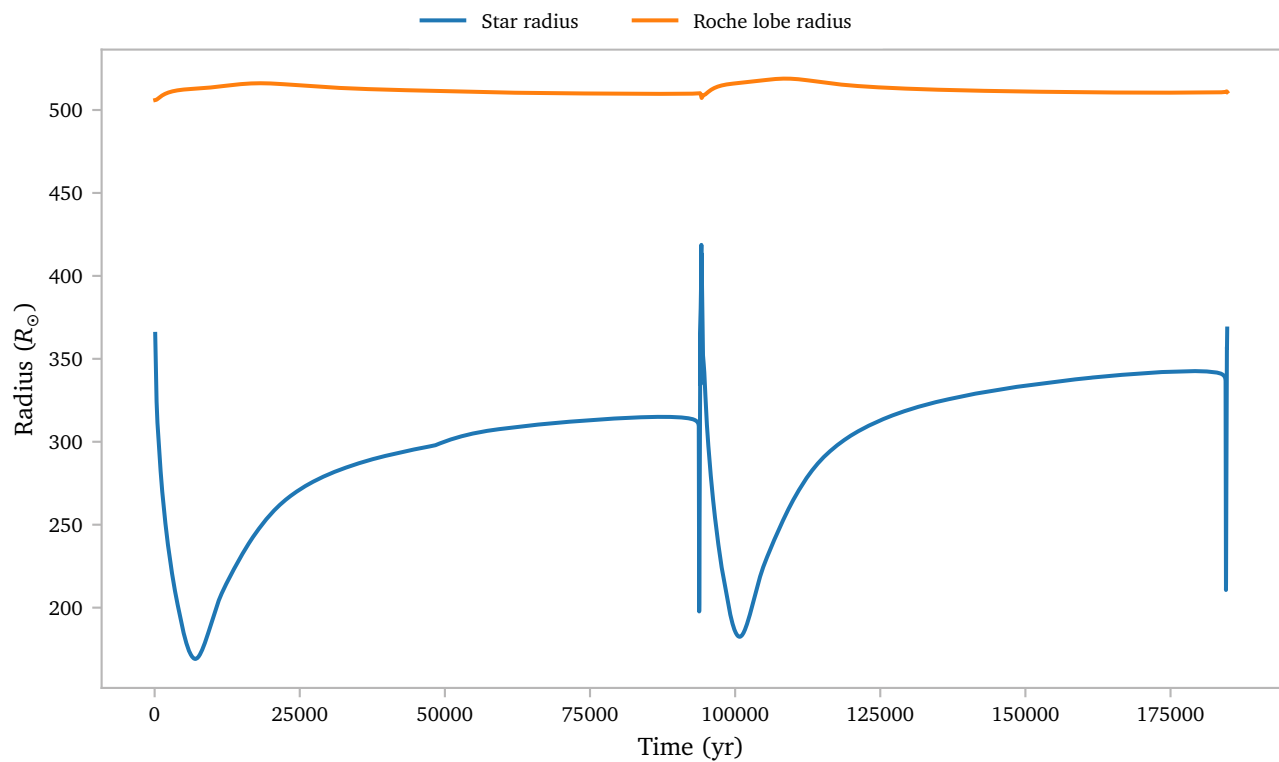
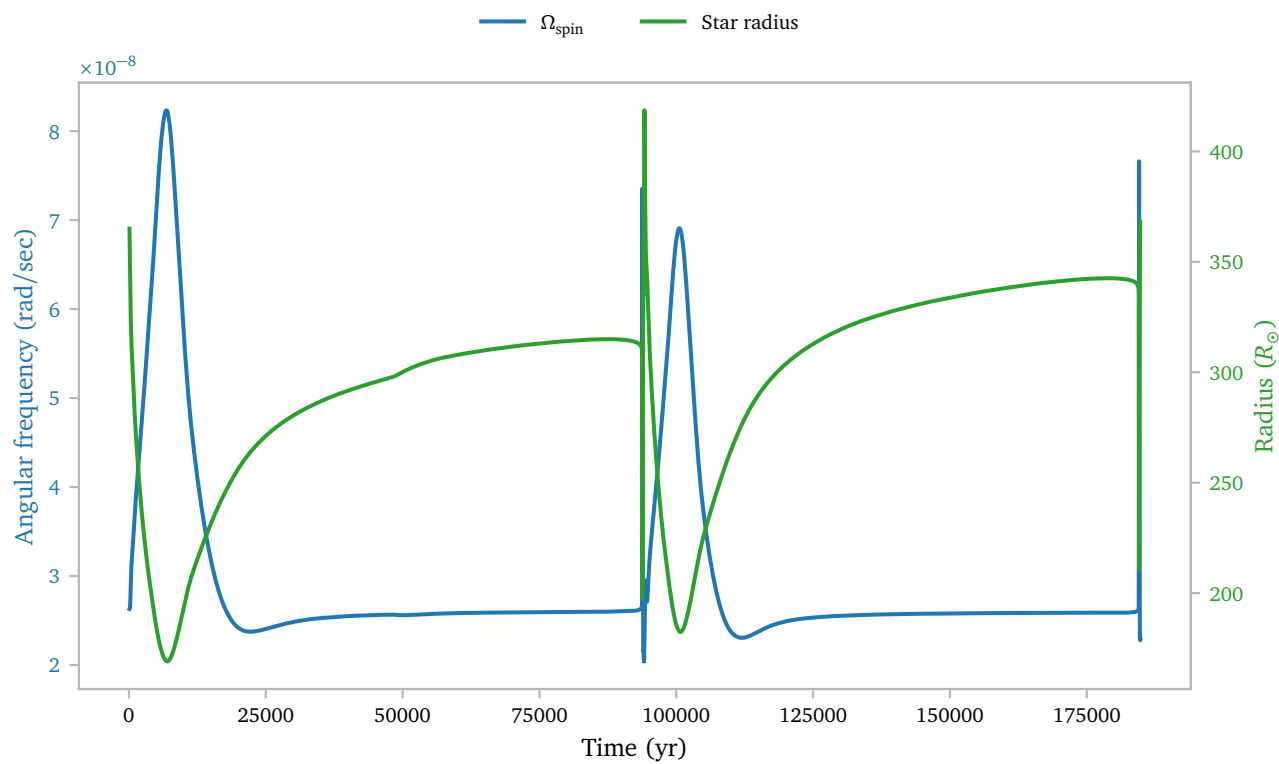
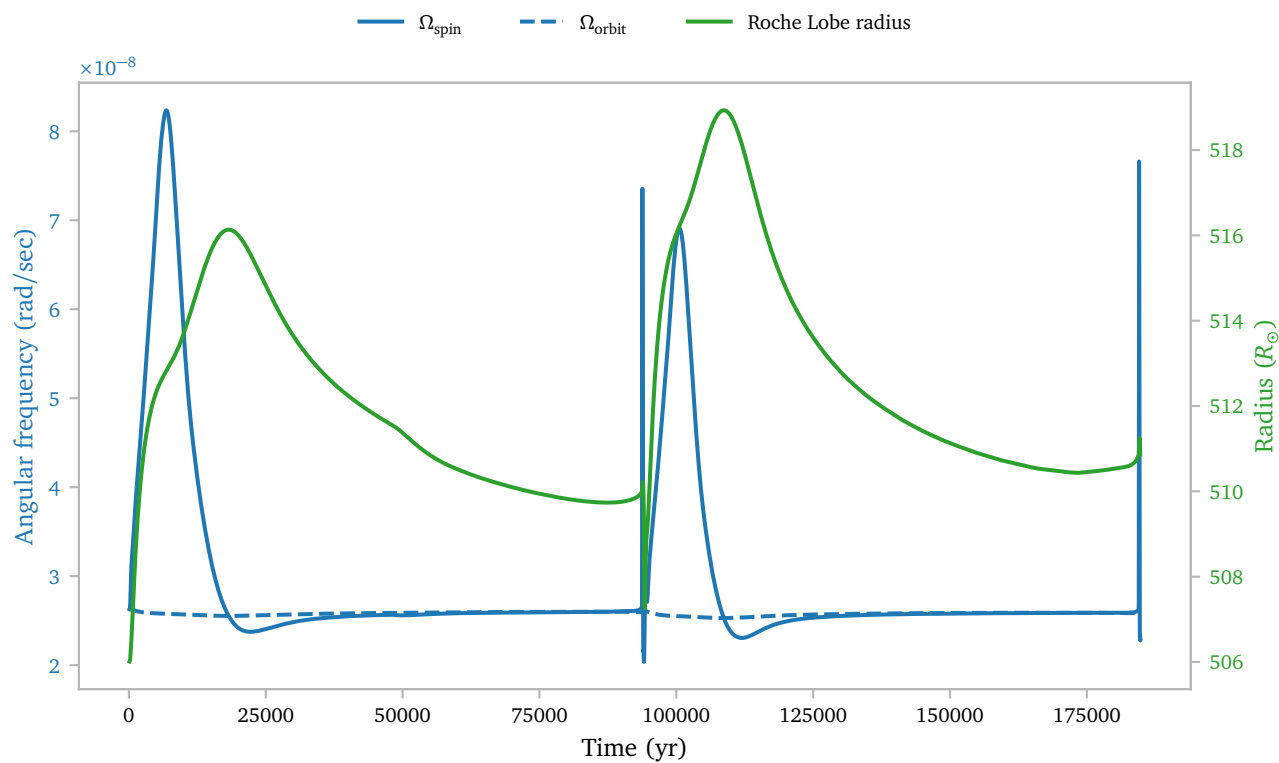
real(dp) function compute_t_conv(s, M_conv, R_conv) result(t_conv)
implicit none

type(star_info), pointer :: s
real(dp) M_conv, R_conv, L

t_conv = 0.4311* (3 * M_conv * R_conv**2 / (s% photosphere_L))**(1/3) * secyer

end function compute_t_conv
```

Below, I show the results of a test run:



References

Goldreich, P. & Nicholson, P. D. (1989) *Tidal Friction in Early-Type Stars*. [ApJ342, 1079](#).

Hurley, J. R., Tout, C. A., & Pols, O. R. (2002) *Evolution of binary stars and the effect of tides on binary populations*. [MNRAS329, 897](#).

Hut, P (1981) *Tidal evolution in close binary systems*.. [A&A99, 126](#).

Preece, H. P., Hamers, A. S., Neunteufel, P. G., Schaefer, A. L., & Tout, C. A. (2022) *The Equilibrium Tide: An Updated Prescription for Population Synthesis Codes*. [ApJ933, 25](#).

d. Additional plots

